

Week 2: Product and Category Management System

Introduction

The Product and Category Management System is designed to maintain information about skincare products and their respective categories. In a skincare clinic or product store, different products belong to different categories such as moisturizers, face washes, sunscreens, serums, and creams. The system helps organize product details, track stock availability, manage pricing, and generate useful reports. By storing all information in a database, product management becomes easier, faster, and more accurate.

1. Design Product and Category Tables

Category Table

```
SQL> CREATE TABLE Category (    category_id NUMBER(5) PRIMARY KEY,    category_name VARCHAR2(50) NOT NULL UNIQUE,    description VARCHAR2(100));
```

The Category table stores information about different categories of skincare products. Each category is assigned a unique Category ID, which helps identify and manage categories separately. Information such as category name and description is maintained in this table.

Product Table

```
SQL> CREATE TABLE Product (    product_id NUMBER(5) PRIMARY KEY,    product_name VARCHAR2(50) NOT NULL,    category_id NUMBER(5),    price NUMBER(10,2) CHECK (price > 0),    stock_quantity NUMBER(5) CHECK (stock_quantity >= 0),    CONSTRAINT fk_category FOREIGN KEY (category_id)    REFERENCES Category(category_id));

Table created.
```

The Product table stores details about skincare products available in the system. Each product is assigned a unique Product ID and is linked to a category through Category ID. The table stores product information such as product name, price, and stock quantity.

2. Define Relationships and Constraints

Relationship Between Category and Product

A category can contain multiple products, but a product can belong to only one category. Therefore, the relationship between Category and Product is a **One-to-Many (1:M)** relationship.

For example:

- The Moisturizers category can contain Aloe Moisturizer, Vitamin Moisturizer, and Herbal Moisturizer.
- However, Aloe Moisturizer can belong only to the Moisturizers category.

Primary Key

A Primary Key is a field that uniquely identifies each record in a table.

- **Category_ID** acts as the Primary Key in the Category table.
- **Product_ID** acts as the Primary Key in the Product table.

Primary keys ensure that no duplicate records exist in the database.

Foreign Key

A Foreign Key establishes a relationship between two tables.

- **Category_ID** in the Product table acts as a Foreign Key.
- It references **Category_ID** in the Category table.

This relationship ensures that every product is associated with a valid category.

NOT NULL Constraint

The NOT NULL constraint ensures that important fields cannot contain empty values.

Examples:

- Category Name must always be provided.
- Product Name must always be provided.

This prevents incomplete information from being stored in the database.

CHECK Constraint

The CHECK constraint validates data before it is stored.

Examples:

- Product price must be greater than zero.
- Stock quantity must not be negative.

This helps maintain accurate and meaningful data.

UNIQUE Constraint

The UNIQUE constraint ensures that duplicate values are not allowed.

Example:

- Category Name must be unique.
- Two categories cannot have the same category name.

This improves data consistency and avoids confusion.

3. Insert Values

```
SQL> INSERT INTO Category VALUES (1,'Moisturizers','Skin Hydration Products');
1 row created.

SQL> INSERT INTO Category VALUES (2,'Face Wash','Cleansing Products');
1 row created.

SQL> INSERT INTO Category VALUES (3,'Sunscreen','Sun Protection Products');
1 row created.

SQL> INSERT INTO Category VALUES (4,'Serums','Skin Treatment Products');
1 row created.

SQL> INSERT INTO Category VALUES (5,'Creams','Beauty Cream Products');
1 row created.
```

The INSERT operation is used to add records into the Category and Product tables.

Category Records

The following categories are maintained in the database:

- Moisturizers
- Face Wash

- Sunscreen
- Serums
- Creams

Each category contains a description that explains its purpose.

Product Records

```
SQL> INSERT INTO Product VALUES (101,'Aloe Moisturizer',1,350,50);
1 row created.

SQL> INSERT INTO Product VALUES (102,'Gentle Face Wash',2,250,40);
1 row created.

SQL> INSERT INTO Product VALUES (103,'SPF 50 Sunscreen',3,500,30);
1 row created.

SQL> INSERT INTO Product VALUES (104,'Vitamin C Serum',4,700,20);
1 row created.

SQL> INSERT INTO Product VALUES (105,'Night Cream',5,450,25);
1 row created.
```

```
SQL> INSERT INTO Product VALUES (106,'Herbal Moisturizer',1,300,20);
1 row created.

SQL> INSERT INTO Product VALUES (107,'Vitamin Moisturizer',1,400,15);
1 row created.

SQL> INSERT INTO Product VALUES (108,'Hydra Moisturizer',1,450,10);
1 row created.

SQL> INSERT INTO Product VALUES (109,'Daily Moisturizer',1,350,25);
1 row created.

SQL> INSERT INTO Product VALUES (110,'Glow Moisturizer',1,500,12);
1 row created.
```

The Product table contains products such as:

- Aloe Moisturizer
- Gentle Face Wash
- SPF 50 Sunscreen
- Vitamin C Serum
- Night Cream

Each product is linked to a category and includes information about price and stock availability.

4. CRUD Operations

CRUD stands for Create, Read, Update, and Delete. These operations are used to manage data stored in the database.

Create Operation

The INSERT statement is used to add new product records into the Product table.

Example:

A new skincare product such as Rose Face Serum can be added to the database.

Read Operation

The SELECT statement is used to retrieve information from the database.

Users can view:

- All products
- All categories
- Product details
- Stock information

Update Operation

```
SQL> UPDATE Product SET price = 750 WHERE product_id = 104;  
1 row updated.
```

The UPDATE statement is used to modify existing records.

Example:

- Updating the price of Vitamin C Serum from ₹700 to ₹750.
- Updating stock quantity when new products arrive.

Delete Operation

```
SQL> DELETE FROM Product WHERE product_id = 105;  
1 row deleted.
```

The DELETE statement is used to remove unwanted records.

Example:

- Removing a discontinued skincare product from the Product table.

CRUD operations make the system flexible and easy to maintain.

5. Generate Category-wise Product Reports

Display All Products Under Each Category

```
SQL> SELECT c.category_id, c.category_name, p.product_id, p.product_name, p.price, p.stock_quantity FROM Category c
JOIN Product p ON c.category_id = p.category_id;
```

CATEGORY_ID	CATEGORY_NAME	PRODUCT_ID
PRODUCT_NAME	PRICE	STOCK_QUANTITY
1	Moisturizers	101
Aloe Moisturizer	350	50
2	Face Wash	102
Gentle Face Wash	250	40
3	Sunscreen	103
SPF 50 Sunscreen	500	30
4	Serums	104
Vitamin C Serum	750	20

This report displays all products along with their corresponding categories. It helps users understand the products available in each category and improves product organization.

Count Number of Products in Each Category

```
SQL> SELECT c.category_name, COUNT(p.product_id) AS total_products FROM
Category c LEFT JOIN Product p ON c.category_id = p.category_id GROUP BY
c.category_name;
```

CATEGORY_NAME	TOTAL_PRODUCTS
Face Wash	1
Serums	1
Creams	0
Moisturizers	1
Sunscreen	1

This report calculates the total number of products available under every category. It helps management understand product distribution and identify categories with fewer products.

Find Highest-Priced Product in Each Category

```
SQL> SELECT c.category_name, MAX(p.price) AS highest_price FROM Category
c JOIN Product p ON c.category_id = p.category_id GROUP BY c.category_n
ame;
```

CATEGORY_NAME	HIGHEST_PRICE
Face Wash	250
Serums	750
Moisturizers	350
Sunscreen	500

This report identifies the product with the maximum price in each category. It helps analyze premium products and compare pricing strategies across categories.

Display Categories Having More Than three Products

```
SQL> SELECT c.category_name, COUNT(p.product_id) AS total_products FROM
Category c JOIN Product p ON c.category_id = p.category_id GROUP BY c.ca
tegory_name HAVING COUNT(p.product_id) > 5;
```

CATEGORY_NAME	TOTAL_PRODUCTS
Moisturizers	6

This report displays categories that contain more than five products. It helps identify categories with a large product inventory and supports stock management decisions.

Find Average Product Price Category-wise

```
SQL> SELECT c.category_name, AVG(p.price) AS average_price FROM Category
c JOIN Product p ON c.category_id = p.category_id GROUP BY c.category_n
ame;
```

CATEGORY_NAME	AVERAGE_PRICE
Face Wash	250
Serums	750
Moisturizers	391.666667
Sunscreen	500

This report calculates the average price of products within each category. It helps management understand pricing trends and compare the overall value of products across categories.

THIRUVARASI. S

B.Sc CS WITH AI [S-1]

E25AI146